6. Call `Run()` on the `WebApplication` to start the server and handle requests.

The basic minimal API app shown previously in listing 3.3 was simple enough that it didn't need steps 2 and 4, but otherwise it followed this sequence in its Program.cs file. The following listing extends the default application to add more functionality, and in doing so it uses all six steps.

Listing 3.4 The Program.cs file for a more complex example minimal API

```
using Microsoft.AspNetCore.HttpLogging;

WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddHttpLogging(opts =>                          ❶
    opts.LoggingFields = HttpLoggingFields.RequestProperties);   ❶

builder.Logging.AddFilter(                                       ❷
    "Microsoft.AspNetCore.HttpLogging", LogLevel.Information);   ❷

WebApplication app = builder.Build();

if (app.Environment.IsDevelopment())                             ❸
{
    app.UseHttpLogging();                                        ❹
}

app.MapGet("/", () => "Hello World!");
app.MapGet("/person", () => new Person("Andrew", "Lock"));       ❺

app.Run();

public record Person(string FirstName, string LastName);         ❻
```

❶ You can customize features by adding or customizing the services of the application.

❷ Ensures that logs added by the HTTP logging middleware are visible in the log output

❸ You can add middleware conditionally, depending on the runtime environment.

❹ The HTTP logging middleware logs each request to your application in the log output.

❺ Creates a new endpoint that returns the C# object serialized as JSON

❻ Creates a record type

The application in listing 3.4 configures two new features: